

# Homework 5

March 24, 2025

```
[1]: import numpy as np
import matplotlib.pyplot as plt
```

## 1 1.

To implement RK-2, firstly I'll need to split the second order differential equation  $y''(t) = -g$  into two coupled first order ones as such:

$$\frac{dy}{dt} = v \quad (1)$$

$$\frac{dv}{dt} = -g \quad (2)$$

Which I can then solve simultaneously by defining  $\mathbf{r} = (y(t), v(t))$ .

```
[2]: g = 9.81 # m/s^2

def f(r):
    y = r[0]
    v = r[1]

    fy = v
    fv = -g

    return np.array([fy, fv])

# implement RK-2
def RK2(v0):
    # discretize t grid from 0 to 10
    t_i = 0
    t_f = 10
    n = 1001
    h = (t_f - t_i) / (n - 1)

    t = np.linspace(t_i, t_f, n)

    # create y and v arrays
    y = np.zeros(n)
    v = np.zeros(n)
```

```

y[0] = 0
v[0] = v0

r = np.array([y[0],v[0]])

for i in range(n-1):
    k1 = f(r)
    k2 = f(r+k1*h/2)

    r = r + h*k2

    y[i+1] = r[0]
    v[i+1] = r[1]

return t, y, v

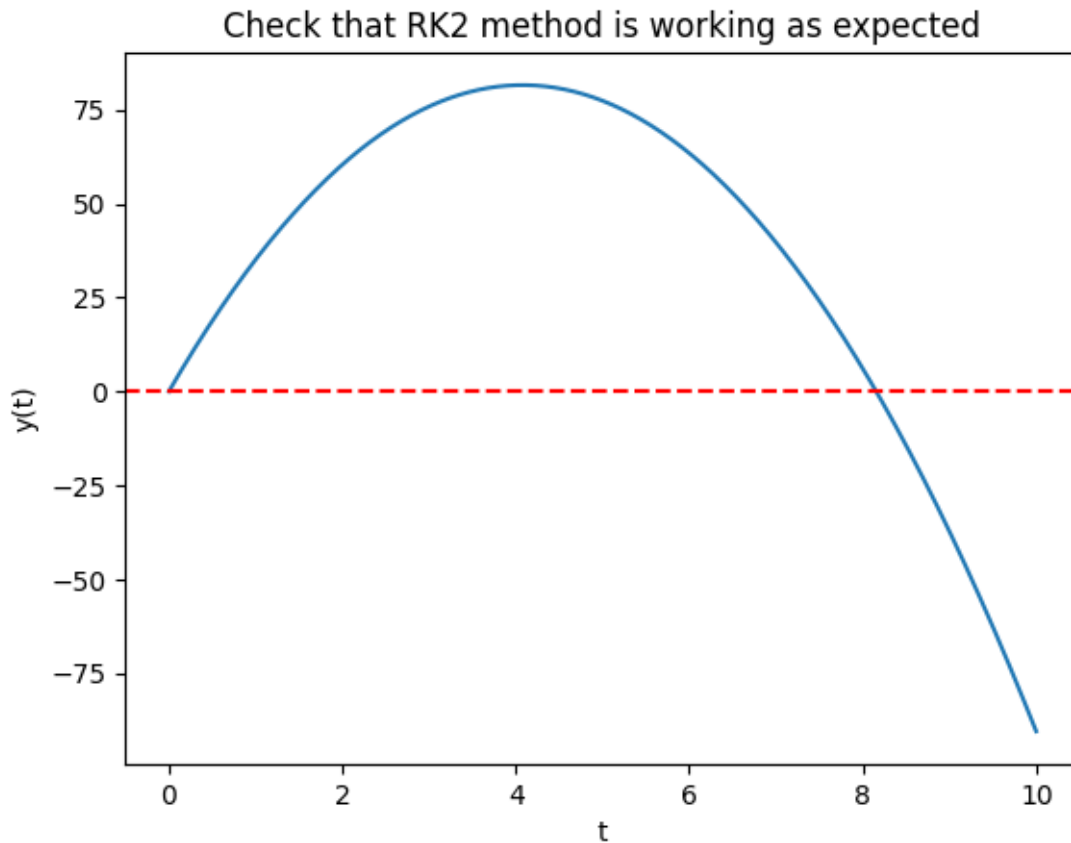
# check the RK2 is working as expected
t, y_check, v_check = RK2(40)

print(v_check[0])
plt.plot(t, y_check)
plt.axhline(y=0, ls='--', c='r')
plt.title('Check that RK2 method is working as expected')
plt.xlabel('t')
plt.ylabel('y(t)')

```

40.0

[2]: Text(0, 0.5, 'y(t)')



The RK2 function seems to be working fine since the resulting function seems to be parabolic.

To implement the secant shooting method, I'll define  $g(\alpha) = u_\alpha(x_{n-1}) - y(x_{n-1})$  where  $\alpha$  is the shooting parameter, in this case the initial velocity, and  $u_\alpha$  is the solution using  $v(0) = \alpha$ . Since  $y(x_{n-1}) = 0$  for this problem, I just need to return  $u_\alpha(x_{n-1})$ .

```
[3]: # define g function and return u(x_{n-1})
def ga(alpha):
    t, y, v = RK2(alpha)
    return y[-1]

# implement secant method to solve for the shooting parameter
def Secant(fun, guess_0, guess_1, tol=1e-8):
    for i in range(1,100):
        g0 = fun(guess_0)
        g1 = fun(guess_1)

        ratio = (guess_1 - guess_0) / (g1 - g0)
        guess_2 = guess_1 - g1*ratio
```

```

        if abs(guess_2-guess_1) < tol:
            break

        guess_0 = guess_1
        guess_1 = guess_2

    return guess_2

# compute the actual initial velocity
v_actual = Secant(ga, 40, 50)

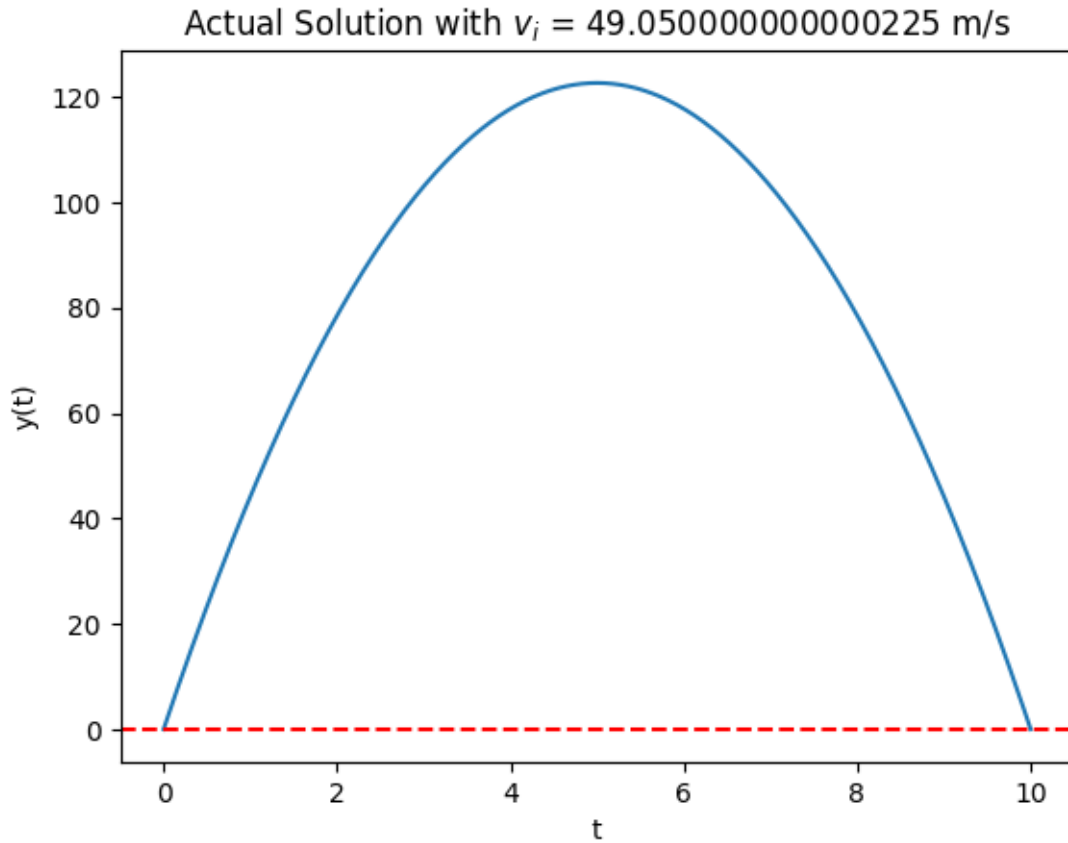
t, y, v = RK2(v_actual)

print(v_actual)
plt.plot(t, y)
plt.axhline(y=0, ls='--', c='r')
plt.title(f'Actual Solution with  $v_i = \{v\_actual\}$  m/s')
plt.xlabel('t')
plt.ylabel('y(t)')

```

49.0500000000000225

[3]: Text(0, 0.5, 'y(t)')



## 2 2.

Setting all the constants to 1, our differential equation looks like

$$\frac{d^2\psi}{dx^2} = (V(x) - E)\psi \quad (3)$$

which I can turn into a coupled system of first order differential equations by allowing  $\psi'(x) = \phi$ .

```
[4]: # define the potential as |x| between -0.5 and 0.5
def V(x):
    if x >= -0.5 and x <= 0:
        return -1*x
    elif x > 0 and x <= 0.5:
        return x
    else:
        return 1e9

# define f
def f2(r, x, E):
    psi = r[0]
```

```

    phi = r[1]

    f_psi = phi
    f_phi = (V(x) - E) * psi

    return np.array([f_psi, f_phi])

# Implement Foward Eurler Method
def FE(E, xi=-0.5, xf=0.5, n=1001):
    h = (xf - xi) / (n-1)

    x = np.linspace(xi,xf,n)

    psi = np.zeros(n)
    phi = np.zeros(n)
    psi[0] = 0 # from boundary value
    phi[0] = 1 # can be any value, determined later

    r = np.array([psi[0], phi[0]])

    for i in range(n-1):
        r = r + h*f2(r,x[i],E)
        psi[i+1] = r[0]
        phi[i+1] = r[1]

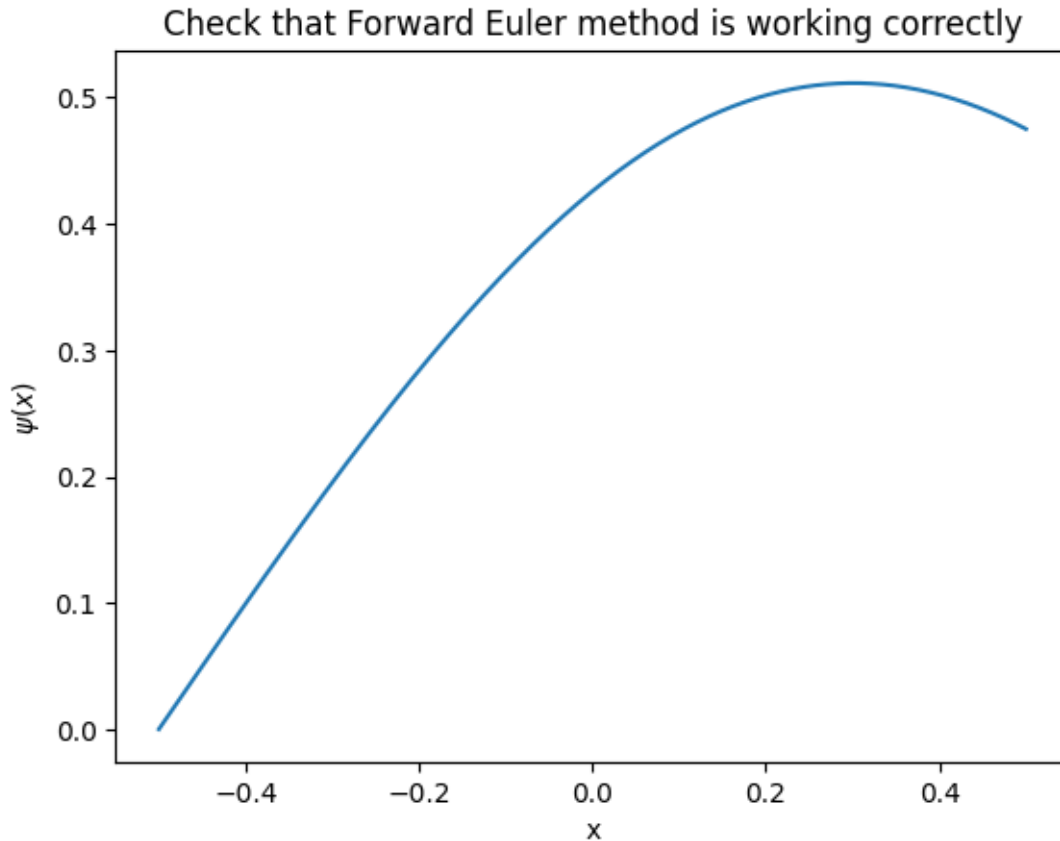
    return x, psi, phi

# check that our Foward Euler function is working as expected
x, psi, phi = FE(4)

plt.plot(x,psi)
plt.title('Check that Forward Euler method is working correctly')
plt.xlabel('x')
plt.ylabel(r'$\psi(x)$')

```

```
[4]: Text(0, 0.5, '$\psi(x)$')
```



I can use the same secant method function I defined in question 1 to implement the shooting method. In this case, the last value in the  $\psi$  array should be 0, so  $g(\alpha)$  will be just be the last value returned from the Forward Euler method.

```
[5]: # define g(alpha) function that I want to find the root of
def ga2(E):
    x, psi, phi = FE(E)
    return psi[-1]

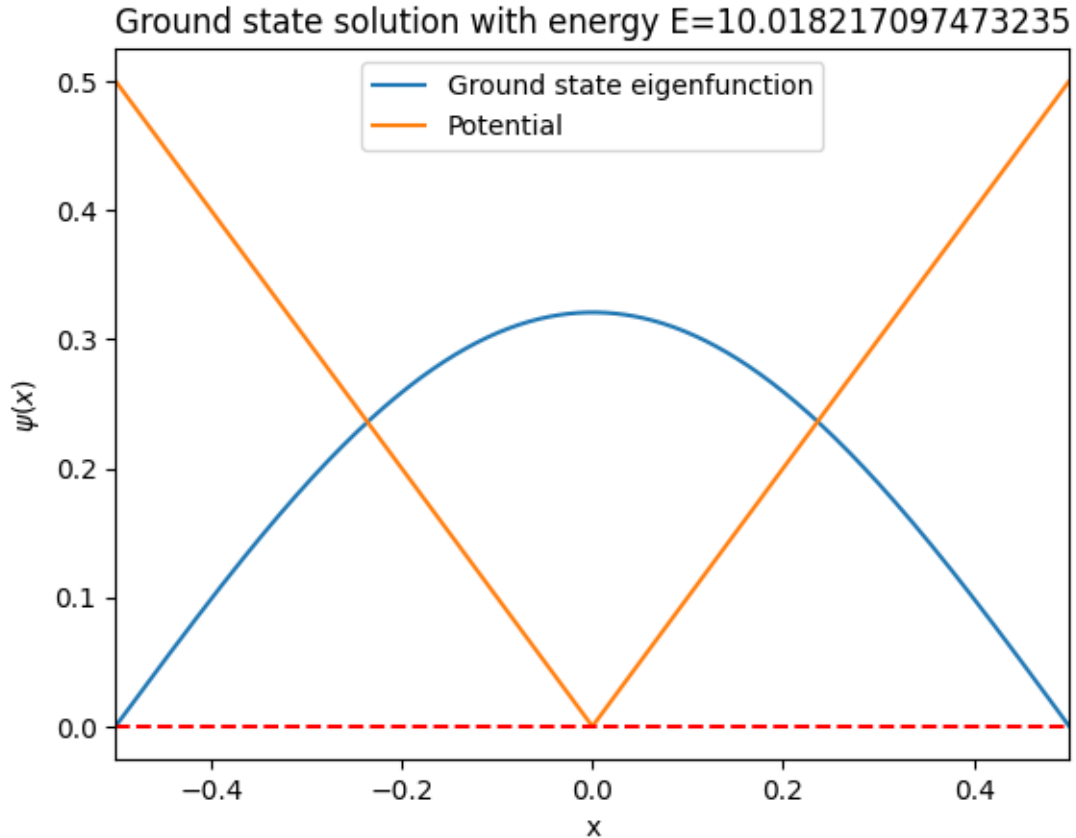
# find the appropriate ground state energy. 4 was too small so I use it and a
# ↪ larger number as my initial guesses
E_actual = Secant(ga2, 4, 10)

# solve for the ground state eigenfunction using the accurate energy
x, psi, phi = FE(E_actual)
```

```
[6]: V = np.vectorize(V)
potential = V(x)
plt.plot(x,psi, label='Ground state eigenfunction')
plt.plot(x,potential, label='Potential')
```

```
plt.axhline(y=0, ls='--', c='r')
plt.xlim(-0.5,0.5)
plt.title(f'Ground state solution with energy E={E_actual}')
plt.xlabel('x')
plt.ylabel(r'$\psi(x)$')
plt.legend()
```

[6]: <matplotlib.legend.Legend at 0x133f6cd01a0>



### 3 3.

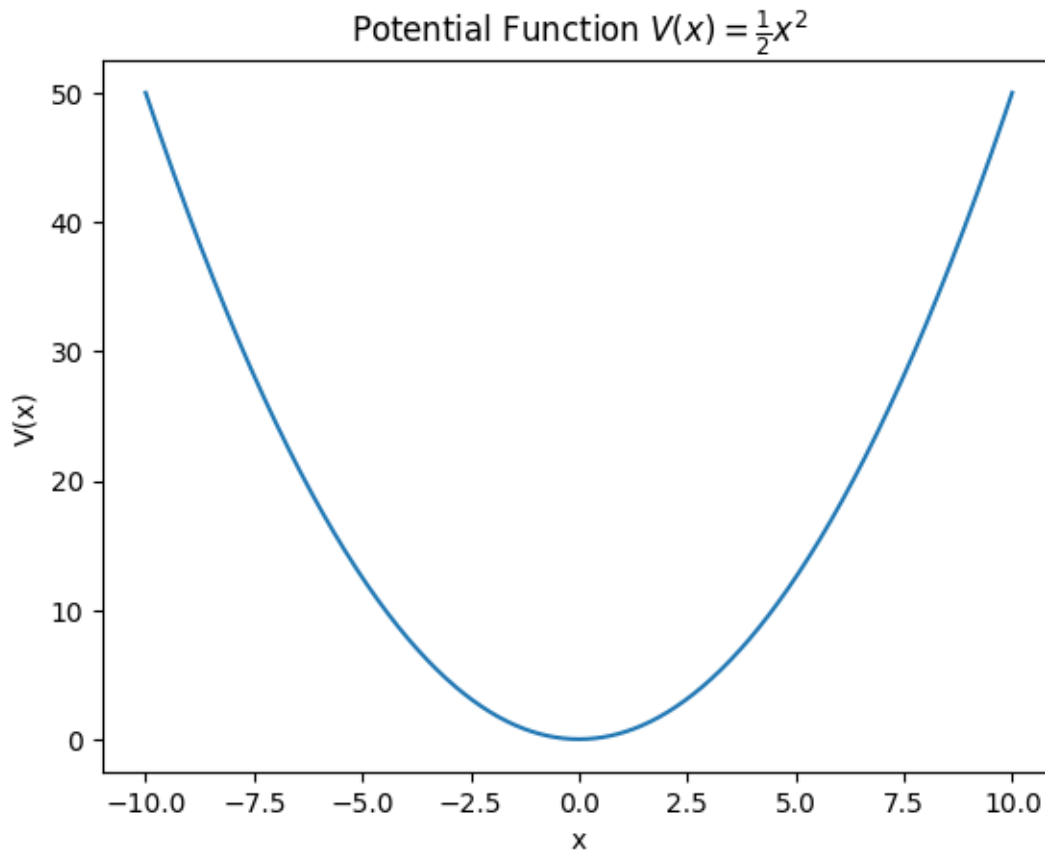
```
[7]: # define  $V(x) = 1/2 x^2$ 

def V3(x):
    return 0.5 * x**2 # can return either single values or an array

# check that potential function works as expected
x3 = np.linspace(-10,10,1001)
plt.plot(x3,V3(x3))
```

```
plt.title(r'Potential Function  $V(x)=\frac{1}{2}x^2$ ')
plt.xlabel('x')
plt.ylabel('V(x)')
```

```
[7]: Text(0, 0.5, 'V(x)')
```



```
[8]: # define x grid
x_i = -10
x_f = 10
n = 1001
dx = (x_f - x_i) / (n - 1)
k = 1 / (2 * dx**2)

x = np.linspace(x_i, x_f, n)

# define the Hamiltonian matrix
Hamiltonian = np.zeros((n-2,n-2))

# define the first row
```

```

Hamiltonian[0][0] = V3(x[1]) + 2*k
Hamiltonian[0][1] = -k

# define the last row
Hamiltonian[-1][-2] = -k
Hamiltonian[-1][-1] = V3(x[n-2]) + 2*k

# define the rest of the hamiltonian matrix
for i in range(1,n-3):
    Hamiltonian[i][i-1] = -k
    Hamiltonian[i][i+1] = -k
    Hamiltonian[i][i] = V3(x[i+1]) + 2*k

# check that the hamiltonian has the expected form
print(Hamiltonian[:4][:4])

```

```

[[ 2549.8002 -1250.         0.         ...      0.         0.         0.         ]
 [-1250.      2549.6008 -1250.         ...      0.         0.         0.         ]
 [   0.      -1250.      2549.4018 ...      0.         0.         0.         ]
 [   0.         0.      -1250.         ...      0.         0.         0.         ]]

```

```

[9]: vals, vecs = np.linalg.eigh(Hamiltonian)
print('First three eigenenergies: ',vals[0:3]) # the exact energies should be 0.
↪5, 1.5, 2.5

```

First three eigenenergies: [0.4999875 1.4999375 2.49983749]

The exact solutions for the Quantum Harmonic Oscillator (with  $\hbar, m$ , and  $\omega_0 = 1$ ) are

$$\psi_n(x) = \pi^{-0.25} (2^n n!)^{-0.5} H_n(x) e^{-\frac{x^2}{2}} \quad (4)$$

where  $H_n(x)$  is the  $n^{\text{th}}$  Hermite polynomial.

```

[10]: import math

def ExactSoln(x,n):
    # define Hermite Polynomials
    H = 1
    if n == 1:
        H = 2 * x
    elif n == 2:
        H = 4*x**2 - 2

    return (1/np.pi)**0.25 * (1/np.sqrt(2**n * math.factorial(n))) * H * np.
    ↪exp(-x**2 / 2)

for i in range(3):
    # create psi vector with boundary values = 0
    psi = np.zeros(n)

```

```

psi[1:-1] = vecs[:,i]

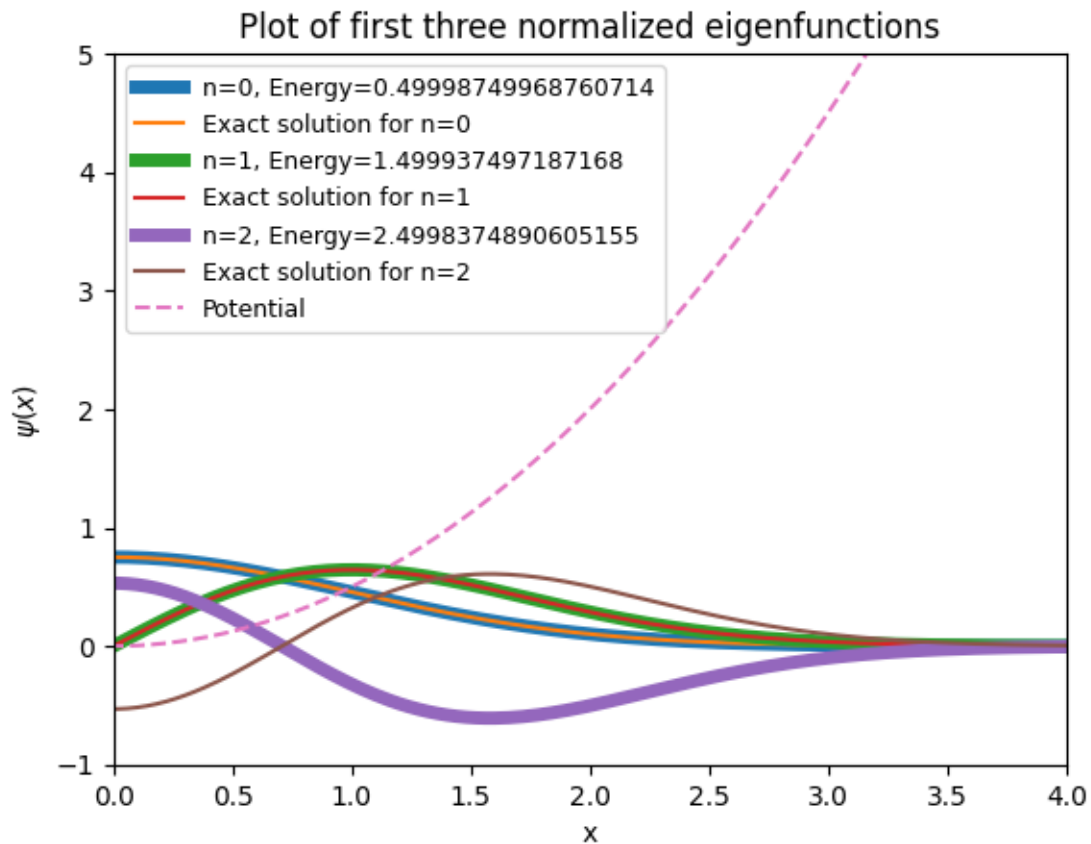
# normalize psi vector
integral = np.trapezoid(psi**2, x, dx)
norm = 1 / np.sqrt(integral)
psi = norm * psi

plt.plot(x,psi, lw=5,label=f'n={i}, Energy={vals[i]}')
plt.plot(x,ExactSoln(x,i), label=f'Exact solution for n={i}')

plt.plot(x,V3(x), ls='--', label='Potential')
plt.legend(fontsize=9)
plt.xlim(0,4)
plt.ylim(-1,5)
plt.title('Plot of first three normalized eigenfunctions')
plt.xlabel('x')
plt.ylabel(r'$\psi(x)$')

```

[10]: Text(0, 0.5, '\$\psi(x)\$')



[ ]: